

MetricLens

Interview Defense & Ownership Document

Sidharth Kriplani

v0.1.0 · pip install metriclens · MIT License · May 2026

Package	metriclens — DataFrame-native metric movement decomposition
Math	Exact algebraic identity: $\text{mix} + \text{rate} + \text{cross} = \text{total}$. No residual.
Metric types	SumMetric, CountMetric, RatioMetric, AverageMetric
Output	JSON, Markdown, HTML — always includes interpretation note
Tests	19 passing — ratio identity, additive sum, edge cases
Not	Not causal inference, not anomaly detection, not significance testing

MetricLens — Interview Defense & Ownership Document

Ground Truth · Build Evidence · Claim Audit · Q&A

Version: 1.0 · May 2026

Author: Sidharth Kriplani

Package: `metriclens` v0.1.0 · MIT License · pip-installable · Open-source

SECTION 0 — WHAT THIS DOCUMENT IS

This document exists because if you put MetricLens on a resume, LinkedIn, or in an interview, you will be asked to defend it. This document ensures every claim you make is traceable to a specific file, formula, or test. Nothing here is inflated. Nothing is borrowed from aspirational roadmap language.

Ground rule: Every claim in this document maps to code that exists in the repo. If something is planned but not built, it is labeled [FUTURE].

SECTION 1 — ONE-LINE TRUTH

MetricLens is a pip-installable Python library that decomposes metric movement into segment contributions, mix effects, rate effects, and cross terms. It is deterministic, DataFrame-native, and honest about what it cannot do.

That sentence is fully defensible. Every word in it corresponds to working code.

SECTION 2 — WHAT YOU BUILT AND WHY

The problem you solved

Every data team eventually builds the same one-off analysis: a metric moved — revenue dropped 14%, conversion rate fell 2.8pp — and someone needs to know *where* it came from and *why*. The typical answer is an ad-hoc notebook that mixes data loading, decomposition math, and formatting into one file, is never reused, and produces inconsistent results across analysts.

No clean pip-installable library existed that:

- Correctly separated mix shift from rate shift with an exact algebraic identity
- Handled edge cases properly (disappeared segments, zero baseline, flat metrics, null dimensions)
- Produced structured JSON/Markdown/HTML output consistently
- Was honest in every output that it reports signals, not causes

You built MetricLens to fill that gap.

Why this is a real engineering decision

The core design choice is treating metric decomposition as an **exact algebraic identity**, not an approximation. The ratio decomposition:

```
total_effect = mix_effect + rate_effect + cross_term
```

holds exactly (up to floating-point) for every segment, across all dimensions, with no residual. This required:

- Correct zero-fill convention for new/disappeared segments
- Explicit cross-term reporting (discarding it breaks the identity)
- Ratio-style decomposition for `AverageMetric` (treating average as numerator/denominator sum)

A simpler implementation would approximate or ignore the cross term. You did not.

SECTION 3 — THE MATH, COLD

You must be able to derive this on a whiteboard.

Additive decomposition

For `SumMetric` and `CountMetric`:

```
segment_delta_s = current_value_s - baseline_value_s  
contribution_pct_s = segment_delta_s / total_delta  
(None if total_delta ≈ 0)
```

All segment deltas sum to `total_delta` exactly. No residual term.

Ratio decomposition

For `RatioMetric` and `AverageMetric`, let segment `s` have:

```
w_b_s = baseline denominator share = denominator_s_baseline / total_denominator_baseline  
w_c_s = current denominator share = denominator_s_current / total_denominator_current  
r_b_s = baseline rate = numerator_s_baseline / denominator_s_baseline  
r_c_s = current rate = numerator_s_current / denominator_s_current
```

Population rates:

```
R_b = Σ_s w_b_s × r_b_s  
R_c = Σ_s w_c_s × r_c_s
```

Segment-level movement decomposed exactly:

```
w_c_s × r_c_s - w_b_s × r_b_s = (w_c_s - w_b_s) × r_b_s ← mix_effect + w_b_s × (r_c_s - r_b_s) ←  
rate_effect + (w_c_s - w_b_s) × (r_c_s - r_b_s) ← cross_term
```

Identity check:

```
Σ_s (mix_effect_s + rate_effect_s + cross_term_s) = R_c - R_b
```

This identity is tested in `tests/test_decomposition.py`: `test_ratio_identity_residual_is_near_zero()`.

What each effect means

mix_effect: Segment changed its share of the denominator. If mobile grew from 40% to 55% of sessions, and mobile has a lower CVR than average, CVR falls even if no individual segment's rate changed. Pure composition shift.

rate_effect: Segment's own rate changed. If paid-search CVR fell from 7.3% to 5.8%, this captures that rate degradation weighted by the segment's baseline volume share.

cross_term: Interaction — the segment both changed in size *and* changed in rate. Usually small but always non-zero when both effects are present. Discarding it means your components don't sum to the total.

Why you report the cross term

Many decompositions silently discard the cross term and attribute it to rate effect or mix effect, producing slight inaccuracies. MetricLens always reports it explicitly because the user deserves to see the exact accounting, not a convenient approximation.

SECTION 4 — EDGE CASES YOU HANDLED

These are tested. Be ready to explain any of them.

Flat metric ($\text{total_delta} \approx 0$)

When $\text{abs}(\text{total_delta}) < 1\text{e-}9$, direction is "FLAT" and `contribution_pct` is `None` for all segments. Dividing by near-zero to get percentage contributions would produce numerically meaningless values.

Test: `test_flat_case_contribution_pct_is_none()`

Zero baseline value

When `baseline_value == 0`, `relative_delta_pct` is `None`. Percentage change from zero is mathematically undefined — reporting $+\infty\%$ would be misleading.

Test: `test_zero_baseline_relative_delta_is_none()`

New and disappeared segments

Zero-fill convention: a segment present in only one period gets $w=0$, $r=0$ in the other.

For a **disappeared** segment:

$\text{cross_term} = (0 - w_b) \times (0 - r_b) = +w_b \times r_b$ (positive, not zero)

This is required. Without it, the ratio identity breaks for segments that vanish.

Test: `test_new_and_disappeared_segments()`

Null dimension values

MetricLens creates an internal working copy of the DataFrame and replaces null dimension values with "`(null)`". The original DataFrame is never modified. Null segments are visible in outputs and contribute to the decomposition correctly.

Additive identity

For `SumMetric`, all `segment_delta` values sum exactly to `total_delta`.

Test: `test_additive_contributions_sum_to_total_delta()`

SECTION 5 — FILES AND WHAT THEY DO

File	Role
src/metriclens/core.py	MetricLens class — period slicing, orchestration, payload assembly
src/metriclens/metrics.py	SumMetric, CountMetric, RatioMetric, AverageMetric — abstract base + four concrete types
src/metriclens/decomposition.py	additive_decomposition(), ratio_decomposition(), null handling, FLAT_EPSILON
src/metriclens/report.py	AnalysisResult — .to_json(), .to_markdown(), .to_html(), .summary(), .segment_contributions()
src/metriclens/profiler.py	DataProfiler — 10 quality checks run automatically on every analyze() call
src/metriclens/exceptions.py	ZeroDenominatorError, DecompositionTypeError
src/metriclens/_version.py	Package version string
src/metriclens/__init__.py	Public API surface: MetricLens, all metric types, exceptions
tests/test_decomposition.py	Ratio identity, additive sum, flat case, zero baseline, new/disappeared segments
tests/test_metrics.py	Per-metric-type compute and compute_by_group correctness
tests/test_profiler_report_demo.py	Data quality checks, output format correctness, demo pipeline
examples/ecommerce_demo.py	End-to-end demo: loads CSV, runs CVR + revenue analysis, writes 6 output files
examples/generate_demo_data.py	Synthetic e-commerce data generator (756 rows, seed=42)
data/ecommerce_demo.csv	Pre-generated demo dataset
outputs/cvr_rca.json/md/html	Real demo outputs committed to repo
outputs/revenue_rca.json/md/html	Real demo outputs committed to repo
.github/workflows/ci.yml	GitHub Actions CI: Python 3.10/3.11/3.12, pytest, ruff
pyproject.toml	Package metadata, build config, dev extras
METHODOLOGY.md	Full algebraic derivation with edge case proofs
CHANGELOG.md	v0.1.0 release notes

SECTION 6 — DATA QUALITY CHECKS

Every `analyze()` call automatically runs 10 quality checks and includes them in the output. You chose to run these automatically — not optionally — because a decomposition on bad data produces correct-looking wrong answers.

Check	What it catches
<code>row_count_baseline</code>	Empty baseline period
<code>row_count_current</code>	Empty current period
<code>date_coverage_baseline</code>	Missing dates within the baseline window
<code>date_coverage_current</code>	Missing dates within the current window
<code>period_length_match</code>	Baseline and current are different lengths (comparing unequal windows distorts mix effects)
<code>null_rate_{dimension}</code>	Null rates per dimension — flags if a large fraction of data has missing segment labels
<code>duplicate_grain</code>	Duplicate rows at the full date × dimension grain — would inflate segment values

SECTION 7 — DESIGN DECISIONS YOU CAN DEFEND

Why additive vs ratio decomposition

Additive metrics (revenue, orders as a count) decompose trivially — each segment's contribution is just its delta. Ratio metrics (CVR = orders/sessions, AOV = revenue/orders) need the mix/rate/cross split because the population rate is a weighted average of segment rates. Changing segment composition changes the population rate even when no individual segment changes. The two decomposition types are architecturally separate for this reason.

Why `AverageMetric` uses ratio decomposition

An average is `numerator_sum / denominator_count`. This is structurally identical to a ratio metric. `MetricLens` models it this way so that `AverageMetric` correctly separates changes in *what is being averaged* (rate effect) from changes in *the composition of what is being averaged* (mix effect).

Why one dimension at a time

Multi-dimensional decomposition (Shapley attribution across dimensions) is non-trivial — it requires 2^n coalitions for n dimensions. `v0` analyzes each dimension independently. This is honest: analyzing `channel` and `device` separately gives you the marginal view of each. Shapley attribution across both simultaneously is deferred to `v1`. This limitation is documented in `METHODOLOGY.md` and should be acknowledged proactively.

Why no statistical significance

`v0` is a decomposition library, not a hypothesis testing library. Significance testing on metric movements requires bootstrap resampling or analytical variance estimates for ratio metrics — non-trivial and scope-expanding. `v1` will add optional bootstrap confidence intervals. Omitting them from `v0` was a scope decision, not an oversight.

Why the interpretation note is mandatory

Every output format — JSON, Markdown, HTML — always includes the interpretation note stating that MetricLens reports signals, not causes. This is not defensive boilerplate. It is an architectural decision: a library that returns segment contributions without this context will be misused by analysts who treat contribution as causation. The note is baked into `MANDATORY_INTERPRETATION_NOTE` in `report.py` and cannot be suppressed.

SECTION 8 — INTERVIEW Q&A

Q: What is mix effect and why does it matter?

Mix effect captures the change in a segment's share of the denominator volume. If mobile grew from 40% to 55% of sessions, and mobile has a lower CVR (5%) than desktop (8%), the population CVR falls purely because of composition shift — even if neither segment's own rate changed. This is the mix effect. Without separating it from rate effect, you'd incorrectly conclude that conversion rates fell when they didn't.

Q: Walk me through the ratio decomposition math.

Start with population rate as weighted sum of segment rates: $R = \sum w_s \times r_s$. The movement from baseline to current for segment s is $w_c \times r_c - w_b \times r_b$. Add and subtract $w_c \times r_b$ and $w_b \times r_c$ to factorize: mix effect is $(w_c - w_b) \times r_b$, rate effect is $w_b \times (r_c - r_b)$, cross term is $(w_c - w_b) \times (r_c - r_b)$. These three sum exactly to the segment's total movement, and summing across all segments gives $R_c - R_b$ exactly.

Q: What is the cross term and why don't you drop it?

The cross term is the interaction between volume change and rate change. It is usually small but never exactly zero when both effects are present. Dropping it means the three components don't sum to the total — you have a residual you're silently attributing somewhere else. I report it explicitly so the output is an exact accounting identity, not an approximation.

Q: What happens when a segment disappears between periods?

I use zero-fill convention: the disappeared segment gets $w=0$, $r=0$ in the current period. The cross term becomes $(0 - w_b) \times (0 - r_b) = +w_b \times r_b$ — positive, not zero. This is required for the identity to hold. I also label that segment `segment_status = "disappeared"` so it's visible in the output.

Q: What's the limitation of analyzing one dimension at a time?

If mobile AND paid-search both declined, analyzing channel and device separately gives you marginal effects of each. But if the joint effect — paid-search mobile specifically — is the real story, the single-dimension analysis won't capture it directly. Multi-dimensional attribution via Shapley values handles this properly, but requires 2^n coalitions and is deferred to v1. The v0 analysis gives you the right places to look, even if it can't pinpoint joint interactions.

Q: Why no p-values or confidence intervals?

v0 is decomposition, not hypothesis testing. Adding significance testing correctly for ratio metrics requires delta method variance estimation or bootstrap resampling. Doing it wrong (naive binomial CI on CVR) produces incorrect intervals. I scoped it out of v0 and it's on the v1 roadmap. I can explain the delta method approach if you want to go into it.

Q: What's the difference between MetricLens and a standard BI pivot table?

A pivot table shows you the numbers. MetricLens attributes the movement — it tells you not just that paid-search went from 7.3% to 5.8% CVR, but how much of the population-level CVR change that rate decline caused (rate effect), whether

paid-search also changed in volume share (mix effect), and what the exact cross-term interaction is. Those are distinct analytical questions and the decomposition answers them algebraically.

Q: Why open source? Why not keep it internal?

The problem it solves — metric movement decomposition — is universal. Every data team writes this from scratch. Making it a public library means it can be shared, improved, and validated by others. It also creates a forcing function for correctness: if the math is wrong, someone will file an issue.

Q: What would you change if you had another week?

CLI (`metriclens analyze --config analysis.yaml`), Shapley attribution across dimensions, and bootstrap confidence intervals for ratio metrics. The CLI is a quality-of-life improvement. Shapley and bootstrap intervals are the two most important analytical gaps in v0.

Q: How do I know the decomposition is correct?

Three ways. First, the ratio identity test: `test_ratio_identity_residual_is_near_zero()` asserts the residual is within floating-point precision of zero. Second, the additive sum test: `test_additive_contributions_sum_to_total_delta()` asserts all segment deltas sum to the total. Third, METHODOLOGY.md contains the full algebraic derivation including the disappeared-segment cross-term proof — the implementation was written to match the derivation, not the other way around.

SECTION 9 — DO NOT OVERCLAIM

Hard do-not-say list

Do Not Say	Why	Say Instead
"MetricLens finds the root cause"	It finds the largest contributor, not the cause	"MetricLens identifies the strongest investigation signal"
"MetricLens is statistically significant"	No hypothesis testing in v0	"MetricLens is deterministic decomposition — significance testing is v1"
"MetricLens detects anomalies"	No anomaly detection exists	"MetricLens decomposes known movements — it doesn't flag unexpected ones"
"Multi-dimensional attribution"	v0 is one dimension at a time	"Per-dimension decomposition; Shapley attribution is v1"
"Production-deployed"	It's an open-source library, not a deployed service	"pip-installable open-source library"
"Used by X teams/users"	You don't have usage data	Don't claim usage you can't verify

Things that sound weaker but are safer

- "I built it because every team writes this from scratch and there was no clean library for it" ✓

- "The math is an exact algebraic identity — not an approximation" ✓
- "19 tests, all pass, including edge cases the naive implementation gets wrong" ✓
- "It's honest in every output that it reports signals, not causes" ✓

SECTION 10 — YOUR OWNERSHIP NARRATIVE

The strongest version of this story, told in two minutes:

"I built MetricLens because I kept running into the same ad-hoc analysis across different teams — a metric moved and someone needed to know which segments drove it and whether it was a mix shift or a rate shift. The math is standard but nobody had packaged it cleanly. Every team's version had the same gaps: they'd either ignore the cross term and break the algebraic identity, or handle disappeared segments incorrectly, or produce flat-metric outputs that divided by zero. I wrote a proper spec first — documented the derivation, all the edge cases, the output schema — then built the implementation to match the spec. The result is a pip-installable library with four metric types, three output formats, automatic data quality checks, and 19 tests that verify the exact algebraic properties. Every output includes a mandatory interpretation note that this is a signal, not a cause — because a library that returns contributions without that context will be misused."

The three things that make this story strong:

You built it to solve a real, universal problem — not as an exercise

You specified the math before writing code, and the implementation is correct against that spec

You were honest about limitations in the library itself, not just in your pitch

SECTION 11 — CHEAT SHEET

Package: `pip install metriclens` · MIT · Python ≥ 3.10

Core class: `MetricLens(data, date_col, baseline_period, current_period, dimensions)`

Metric types: `SumMetric`, `CountMetric`, `RatioMetric`, `AverageMetric`

Output formats: JSON, Markdown, HTML

Tests: 19, all pass

Additive formula:

```
segment_delta = current - baseline
contribution_pct = segment_delta / total_delta
```

Ratio formula:

```
mix_effect = (w_c - w_b) * r_b
rate_effect = w_b * (r_c - r_b)
cross_term = (w_c - w_b) * (r_c - r_b)
total_effect = mix + rate + cross ← exact identity
```

Key edge cases:

- Flat metric → `contribution_pct = None`
- Zero baseline → `relative_delta_pct = None`
- Disappeared segment `cross_term = +w_b * r_b` (positive)
- Nulls → filled with `"(null)"` in working copy only

What it is not: causal inference, anomaly detection, significance testing, BI dashboard, ML model

One-sentence answer to "what does it do":

"It decomposes how much of a metric's movement came from each segment, and for ratio metrics it separates mix shift from rate shift — as an exact algebraic identity."

MetricLens Interview Defense Document v1.0 · Sidharth Kriplani · May 2026

Built on public-domain math. Every claim maps to a file in the repo.